Pen-DHRL-TD: Teacher-Distilled Deep Hierarchical Reinforcement Learning for Autonomous Penetration Testing via Locally-Hosted LLM Subgoal Supervision

NASimEmu-enhanced Project

July 16, 2026

Abstract

We present Pen-DHRL-TD, a deep hierarchical reinforcement learning (DHRL) agent for autonomous network penetration testing in the NASimEmu simulator, augmented with offline supervision from a locally-hosted large language model (LLM) acting as a *teacher* over the agent's high-level subgoal space. The agent (`NASimNetDHRL`) factorizes its policy into a recurrent graph-neural-network *manager* that selects one of eight named semantic subgoals (e.g. `GAIN_INITIAL_ACCESS`, `ESCALATE_PRIVILEGE`) and persists it for a learned horizon, and a goal-conditioned graph-attention *worker* that emits low-level scan/exploit/pivot actions. A locally-hosted, free, open-weight model (`qwen2.5:3b-instruct`, served via Ollama) is queried *offline only* – never inside the training loop's forward pass – against sanitized, partially-observable state summaries collected by a triggered-sampling data-collection pass, producing a provenance-tracked dataset of (state, goal, confidence) triples. This dataset drives a four-stage training schedule: (1) supervised warm-start of the subgoal head, (2–3) joint PPO training with an exponentially-decaying confidence-weighted distillation loss, and (4) a guaranteed reinforcement-learning-only tail. We report the full architecture, teacher pipeline, training algorithm, and a live explainability mechanism that exposes the manager's subgoal-selection histogram and distillation diagnostics every epoch. On a 12-subnet, 72–96-host dynamic corporate scenario family, we report end-to-end training dynamics, a systematic class-imbalance issue discovered via dataset audit, a checkpoint-selection pitfall uncovered during held-out evaluation, and cross-scenario generalization results against both a metric-frozen checkpoint of the same run and an independently trained, non-distilled Pen-DHRL baseline of identical architecture and parameter count, on which the distilled agent comes out ahead on every tested scenario and every reported metric.

Contents

1 Introduction

Autonomous penetration testing is naturally hierarchical: a human operator does not reason step-by-step over individual port scans and exploit attempts, but over higher-level intentions – "enumerate this host", "escalate on that foothold", "reduce detection risk before proceeding". Flat reinforcement-learning policies trained directly on primitive actions must rediscover this structure from reward alone, which is slow and sample-inefficient on large, sparse-reward network topologies. Pen-DHRL-TD addresses this with an explicit two-level hierarchy: a *manager* that commits to a persistent semantic subgoal, and a *worker* that acts under that subgoal's guidance.

The manager's subgoal space is, by construction, an arbitrary learned embedding bank with no inherent meaning – goal index 3 is only "escalate privilege" because the worker's policy gradient happens to shape it that way. This project's central idea is to bind those otherwise-anonymous subgoal slots to a fixed, human-readable ontology (Section 4) and use an LLM, queried entirely offline against sanitized state summaries, as a cheap source of weak supervision over *which* named goal a reasonable planner would pick at a given state. This is emphatically **not** an LLM-in-the-loop agent: the teacher is never queried during environment interaction or during the PPO forward/backward pass. It is queried once, offline, to build a static dataset, exactly as a human-labeled dataset would be built, and the resulting labels are distilled into the manager's subgoal head via an auxiliary supervised loss that runs alongside – never inside – the reinforcement-learning update.

1.1 Contributions

- A hierarchical graph-policy architecture (`NASimNetDHRL`) with a recurrent GNN manager, a learned subgoal bank, a persistence ("switch") gate, and a goal-conditioned graph-attention worker (Section 3).
- A fixed 8-way semantic goal ontology and an offline LLM-teacher pipeline (state sanitization → JSON-schema-constrained local LLM query → anti-hallucination validation → provenance-tracked dataset) that produces confidence-weighted labels without ever exposing hidden simulator state to the teacher (Sections 4–5).
- A triggered-sampling data-collection procedure that queries the teacher only at strategically informative moments (episode start, new subnet, first compromise, privilege change, IDS alert, repeated failure, periodic fallback, high manager entropy) under a diversified mixture of rollout policies (Section 6).
- A four-stage training schedule combining supervised warm-start, decaying joint distillation, and a guaranteed RL-only tail, plus an optional potential-based reward-shaping channel over the same semantic goal space (Sections 7–9).
- A live, per-epoch *explainability* mechanism that prints the manager's realized subgoal-selection distribution and distillation diagnostics in human-readable form during training – the mechanism that, in this project's own training run, first exposed a subgoal-selection collapse later confirmed through held-out evaluation (Section 10).
- An end-to-end empirical study on a 100-host dynamic corporate scenario family: dataset audit statistics, training dynamics, a checkpoint-selection pitfall, and cross-scenario generalization results against both a same-run checkpoint and an independently trained, non-distilled baseline (Section 12).

2 Environment and Task

The agent operates in NASimEmu, a partially-observable network-attack-simulation gym environment. An episode consists of a randomly-instantiated corporate network; the agent observes only what it has actively discovered (host existence, OS, services, processes, access level, IDS detection level) and must chain scan/exploit/privilege-escalation actions to compromise hosts, subject to a fixed per-episode step budget.

2.1 Scenario family

All experiments use the `corp_100hosts_dynamic.v2` scenario family: 12 labeled subnets (DMZ, user, service, db, dev, app ×3, web, office, research, backups) connected in a fixed topology matrix, each instantiated at reset with a random host count in $[6, 8]$ (72–96 hosts total per episode). Per-subnet sensitive-host probability ranges from 0.0 (DMZ/user) to 1.0 (db), giving a non-uniform, subnet-dependent value landscape. A 32-entry extended service catalog and a process catalog (each tagged with an OS) are drawn per host; three services (`mysql`, `postgres`, `mssql`) are up-weighted on sensitive hosts. Five concrete scenario instances are used across this study – `base`, `varA`, `varB`, `test`, `bridge` – which share topology and service/process catalogs but differ in per-exploit/privesc *success probability* and *cost*, giving a controllable difficulty gradient (Table 1).

Table 1: Mean exploit success probability across the 8-entry exploit catalog, per scenario instance (lower = harder). Costs increase in the same order.

Scenario	Role	Mean exploit prob.
`corp_100hosts_dynamic` (base)	training	≈ 0.79
`corp_100hosts_dynamic_varA`	training	≈ 0.63
`corp_100hosts_dynamic_varB`	training	≈ 0.58
`corp_100hosts_dynamic_bridge`	held-out eval	≈ 0.66
`corp_100hosts_dynamic_test`	held-out eval	≈ 0.53 (hardest)

2.2 Observation and action space

The agent consumes a `graph_v2` observation: nodes are either subnets or hosts (a leading type bit distinguishes them), host nodes carry an 80-dimensional feature vector (discovery/reachability/-compromise/access flags, per-service and per-process booleans, IDS detection level and threshold, sensitivity-irrelevant fields only – see Section 5 for the same partial-observability constraint applied to the LLM teacher), and an 8-dimensional sinusoidal positional encoding is appended per node. Edges follow host reachability. The action space is a (target node, discrete action id) pair over 18 action types (4 fixed scan actions plus the scenario's exploit/privilege-escalation catalog), masked so that subnet nodes and graphs with no valid host targets cannot be selected. An explicit terminate action (`-use_a_t`) lets the agent end an episode early rather than exhausting the 400-step budget.

3 Hierarchical Policy Architecture

`NASimNetDHRL` (212,640 parameters at the hyperparameters used in this study) is a manager–worker hierarchical graph policy, structurally inspired by FeudalNet- and GNN-LSTM-style architectures adapted to a variable-size graph observation.

3.1 Manager

Node features (with positional encoding) are linearly embedded ($\mathbb{R}^{88} \to \mathbb{R}^{64}$) and passed through a 3-iteration multi-message-passing GNN with a recurrent global "last-GRU" readout (`MultiMessagePassingWithGlobalNodeAndLastGRUGlobal`), producing per-node embeddings x_{mgr} and a single recurrent global summary $x_{\text{global}} \in \mathbb{R}^{64}$ per graph (episode). x_{global} is layer-normalized and additively biased by an IDS-aware projection of the graph's mean detection level/threshold/multiplier (`ids_projection`), mirroring a FeudalGTM-style stealth bias: the manager's context is nudged by how "hot" the network currently looks, encouraging (without hard-constraining) detection-averse planning.

3.2 Subgoal selection and persistence

A subgoal head $\pi_{\text{goal}}(\cdot \mid x_{\text{global}})$ (2-layer MLP $\to$ 8 logits) defines a categorical distribution over the K=8 subgoal slots. A separate *switch* gate $\sigma_{\text{switch}}(x_{\text{global}}) \in (0, 1)$ (sigmoid MLP) is sampled as a Bernoulli *persistence-override* signal each step: the manager only re-samples a new subgoal when either (a) the previous subgoal's horizon counter has reached zero, or (b) the switch gate fires. This is what makes subgoals *temporally extended* rather than re-selected every step – purely reactive per-step goal selection would give the worker no stable target to condition a multi-step plan on. Once selected, subgoal index k indexes a learned embedding `goal_bank`$[k] \in \mathbb{R}^{128}$, refined by concatenation with x_{global} through a small MLP (`goal_refine`) to produce the state-conditioned goal vector actually broadcast to the worker. The joint action probability (used for PPO's importance ratio) correctly folds in the probability of *whichever* of these three branches produced the current subgoal (forced switch, policy-initiated switch, or persistence) – Section 3's Algorithm 1, lines constructing `subgoal_factor`, give the exact three-way case split.

3.3 Worker

The refined goal vector is broadcast to every node in its graph and concatenated with the manager's per-node embedding; a 2-head graph-attention layer (`GATConv`) followed by layer norm produces goal-conditioned node features, linearly projected to per-node action logits. Illegal targets (subnet nodes, or every node in a graph with no valid host target) are masked to $-\infty$ before a per-graph softmax, and the realized (target, action) pair is sampled by a segmented categorical sample over the flattened, graph-grouped logits.

3.4 Value, termination, and time pressure

A shared context vector (global embedding $\oplus$ goal vector $\oplus$ normalized elapsed-time fraction) feeds both a value head and a termination head (sigmoid). An explicit linear-plus-threshold time penalty is subtracted from the value estimate as the episode approaches its step budget, and termination is additionally gated to require both a minimum elapsed fraction (25% of the step limit) and a non-positive value estimate – this two-sided gate (Algorithm 1, termination block) is what prevents the policy from learning to terminate near-instantly for a cheap zero-cost episode, while still allowing genuinely early, successful termination once real progress has been made.

Algorithm 1 `NASimNetDHRL.forward` – Manager–Worker Hierarchical Policy Step

Require: batch of graph observations $\{s_i\}$, persistent state $(g^{\text{idx}}_{t-1}, \tau_{t-1}, h_{t-1})$ (subgoal index, steps-remaining, GRU hidden state) carried from the previous call

Ensure: actions $\{(v_i, a_i)\}$, value V, joint action probability π, raw actions $(a^{\text{sel}}, \text{term}, g^{\text{idx}}_t)$

$x \leftarrow \text{Embed}(\text{concat}(\text{node\_feats}, \text{PosEnc}))$

$x_{\text{mgr}}, x_{\text{global}} \leftarrow \text{ManagerGNN}(x, \text{edges}, h_{t-1})$ ▷ recurrent global readout

$x_{\text{global}} \leftarrow \text{LayerNorm}(x_{\text{global}}) + \text{IDSProj}(\bar{\phi}_{\text{ids}})$

$\ell_{\text{goal}} \leftarrow \text{SubgoalHead}(x_{\text{global}})$; $\quad p_{\text{goal}} \leftarrow \text{softmax}(\ell_{\text{goal}})$

$\sigma \leftarrow \text{clamp}(\text{SubgoalSwitch}(x_{\text{global}}), \epsilon, 1-\epsilon)$

if $\tau_{t-1} \le 0$ **then** ▷ horizon exhausted: forced re-sample

  need_new $\leftarrow$ true

end if

switch $\leftarrow$ need_new $\vee$ Bernoulli(σ) ▷ sample-mode; argmax test-mode

if switch **then**

  $g^{\text{idx}}_t \sim \text{Categorical}(p_{\text{goal}})$ ▷ argmax at eval time

  $\tau_t \leftarrow$ goal_horizon ▷ reset persistence counter (=4)

else

  $g^{\text{idx}}_t \leftarrow g^{\text{idx}}_{t-1}$; $\quad \tau_t \leftarrow \tau_{t-1} - 1$

end if

$\pi_{\text{subgoal}} \leftarrow$ probability of *whichever* branch above fired ▷ 3-way case: forced / policy-switch / persistence

$\vec{g} \leftarrow \text{GoalRefine}(\text{concat}(\text{goal\_bank}[g^{\text{idx}}_t], x_{\text{global}}))$

$z \leftarrow \text{GAT}(\text{concat}(x_{\text{mgr}}, \vec{g}[\text{batch\_ind}]),\ \text{edges})$

$\ell_{\text{action}} \leftarrow \text{ActionHead}(\text{LayerNorm}(z))$; mask illegal targets to $-\infty$

$a^{\text{sel}} \sim \text{Categorical}(\text{softmax}(\ell_{\text{action}}))$ per graph ▷ segmented sample

$\text{ctx} \leftarrow \text{concat}(x_{\text{global}}, \vec{g}, t/T)$

$V \leftarrow \text{ValueHead}(\text{ctx}) - \text{TimePenalty}(t/T)$

$p_{\text{term}} \leftarrow \text{TerminationHead}(\text{ctx})$

$\text{term} \sim \text{Bernoulli}(p_{\text{term}}) \ \wedge \ (t \ge 0.25\,T) \ \wedge \ (V \le 0)$ ▷ gated early stop

$$\pi \leftarrow \begin{cases} p_{\text{term}} & \text{term} \\ \pi_{\text{action}} \cdot (1 - p_{\text{term}}) \cdot \pi_{\text{subgoal}} & \text{otherwise} \end{cases}$$

return $\{(v_i, a_i)\}$, V, π, $(a^{\text{sel}}, \text{term}, g^{\text{idx}}_t)$

4 Semantic Goal Ontology

The manager's K=8 subgoal-bank slots have no inherent meaning to the architecture – they are exactly as trainable as an unlabeled latent-goal DHRL baseline. Pen-DHRL-TD fixes a versioned index→name binding (Table 2) so that (a) an LLM teacher can be prompted against stable names instead of opaque integers, (b) potential-based reward shaping functions can be hand- or LLM-authored per named goal, and (c) every trained checkpoint stores its ontology version and refuses to load under a mismatched version (`net.py`'s `Net.save`/`load`), preventing a silent semantic mismatch between a checkpoint's learned goal-bank and a later ontology revision.

Table 2: Fixed semantic goal ontology (version 1).

Idx	Name	Description (as given to the LLM teacher)
0	`DISCOVER_SUBNET`	Reveal new reachable subnet structure
1	`ENUMERATE_HOST`	Learn OS/services/processes of a discovered host
2	`GAIN_INITIAL_ACCESS`	Obtain user or root access on a discovered, reachable host remotely
3	`ESCALATE_PRIVILEGE`	Upgrade an already-held user access to root on the same host
4	`PIVOT`	Move toward/through a newly reachable, deeper subnet
5	`CAPTURE_SENSITIVE_HOST`	Prioritize progress on a host that looks high-value
6	`REDUCE_DETECTION`	Prefer a lower-detection-risk action or target over a riskier one
7	`RECOVER_OR_REPLAN`	Abandon a goal that has repeatedly failed and pick a feasible alternative

5 Offline LLM Teacher Pipeline

5.1 Design constraints

The teacher is queried **offline only**: never inside `forward()`, never during environment interaction, never during the PPO backward pass. It is a locally hosted, free, open-weight model (`qwen2.5:3b-instruct`, served via Ollama's `/api/chat` at zero per-call monetary cost, JSON-schema-constrained decoding, temperature 0) – not a paid hosted API, since nothing in this design needs low latency or a live network call during training. The teacher sees only a *sanitized state summary*, never the raw graph tensor or any hidden simulator field: only hosts with `discovered==1` are listed, and the true sensitive-host label (`HostVector.value`/`discovery_value`) is never exposed – exactly the same restraint independently enforced in the potential-shaping functions of Section 8. The teacher must choose exactly one goal from the closed ontology of Table 2, optionally an already-discovered target, and is explicitly instructed never to propose executable text (shell commands, exploit code); a denylist-based validator enforces this on the free-text fields regardless.

5.2 Prompt and output contract

The system prompt fixes the assistant's role and hard constraints; the user prompt embeds the closed goal list with one-line descriptions and the JSON-serialized state summary, and requests a strict-JSON response validated against a JSON schema (`format` passed to Ollama) with required fields `goal`, `horizon` (1–8), `confidence` ($[0, 1]$), `reason_code`, `rationale`, and optional `target_subnet`/`target_host`. Schema-constrained decoding guarantees *shape* only; a separate semantic validator (`llm_teacher/validator.py`) additionally rejects: any target host/subnet id not literally present in the summary (*anti-hallucination*), a target structurally incompatible with the chosen goal (e.g. `ESCALATE_PRIVILEGE` against a host the agent does not already hold user access on), and any of a small denylist of shell/code tokens appearing in the free-text fields. Up to 3 retries are attempted on invalid/unparsable output; the final attempt is recorded with `valid=False` and a

`reject_reason` rather than silently discarded, so rejected records remain available for later audit.

5.3 Optional escalation cascade

A primary/escalation *cascade* variant (`llm_teacher/cascade.py`) queries a primary model on every state and escalates to a second model only on concrete, checkable triggers: an invalid/unparsable primary answer, a `GAIN_INITIAL_ACCESS` goal with no reachable unclaimed host left (an unsatisfiable goal the primary failed to notice), three consecutive failures against the same target that the primary did not route to `RECOVER_OR_REPLAN`, or low self-reported confidence. The escalation call uses a *veto/critique* design: the second model sees the primary's answer and either endorses it (`defensible=true`) or supplies a full, independently-validated replacement – deliberately narrower and cheaper than an independent re-decision, and not trusted automatically just because it was consulted second. This variant is documented here for completeness; the dataset used in Section 12 was collected with the single-model backend only (all 2,235 records attributed to `qwen2.5:3b-instruct`).

5.4 Heuristic teacher (ablation baseline)

A deterministic, network-free rule-based teacher (`llm_teacher/heuristic_teacher.py`) implements a fixed-priority decision tree over the identical sanitized summary (escalate on any held user access → initial-access on any enumerated-but-unclaimed host → enumerate any reachable-but-unscanned host → capture on any root-access host → reduce detection near an IDS threshold → replan on repeated same-target failure → default explore) and is routed through the same validator as the LLM. Its purpose is a controlled ablation ("if simple hand-written heuristics match the LLM's effect, the LLM is not doing anything a much cheaper mechanism could not") and is not the backend used for the results in this report.

6 Triggered Dataset Collection

Labeling every visited state would be both expensive and redundant. Instead, the teacher is queried only at *strategically triggered* points, under a deliberately diversified mixture of rollout (behavior) policies – pure random action selection, an untrained (randomly initialized) `NASimNetDHRL` at zero training cost, or a partially/fully trained checkpoint – so the dataset is not narrowed to only the states a single fixed policy happens to visit (labeling only successful expert trajectories would starve rare-but-important classes such as `ESCALATE_PRIVILEGE`).

Algorithm 2 Triggered State–Action Labeling (Dataset Collection)

Require: scenario, target valid-record count N, rollout policy π_{roll}, teacher backend $\mathcal{T}$, periodic interval P

Ensure: dataset $\mathcal{D}$ of provenance-tracked (state, goal, confidence) records

$n_{\text{valid}} \leftarrow |\{r \in \mathcal{D} : r.\text{valid}\}|$
while $n_{\text{valid}} < N$ and total steps $<$ max_steps **do**
  reset environment; new random episode_id
  label state at **episode start** (always triggered); update n_{valid}
  for $t = 1, \ldots,$ step_limit **do**
    $a \leftarrow \pi_{\text{roll}}(\text{state})$; step environment; observe (s', r, d, info)
    triggered $\leftarrow$ **false**
    if new subnet discovered **then** triggered $\leftarrow$ **true** ▷ trigger: *new_subnet_discovered*
    else if new host compromised **then** triggered $\leftarrow$ **true** ▷ *first_compromise*
    else if access level increased on any known host **then** triggered $\leftarrow$ **true** ▷ *privilege_change*
    else if new IDS threshold crossing **then** triggered $\leftarrow$ **true** ▷ *ids_alert*
    else if last 3 actions: same target, all failed **then** triggered $\leftarrow$ **true** ▷ *three_consecutive_failures*
    else if π_{roll} is net-based **and** $H(\text{subgoal dist.}) > 0.8\, H_{\max}$ **then** triggered $\leftarrow$ **true** ▷ *high_student_entropy*, net-based policies only
    else if $t \bmod P = 0$ **then** triggered $\leftarrow$ **true** ▷ *periodic* fallback
    end if
    if d **then break** ▷ episode ended
    end if
    if triggered **then**
      summary $\leftarrow$ SanitizedSummary(s') ▷ discovered-only, no hidden fields
      record $\leftarrow \mathcal{T}$(summary) ▷ query teacher; validate; retry $\leq 3\times$
      $\mathcal{D} \leftarrow \mathcal{D} \cup \{\text{record}\}$; update n_{valid} if valid
    end if
  end for
end while
return $\mathcal{D}$

Each episode is stamped with a random `episode_id`, later used by `split_dataset.py` to split train/test at the *scenario-instance* level (never placing two steps of the same episode across the boundary) via a deterministic, seed-salted hash of `episode_id` – stable under dataset growth, since it does not depend on shuffling the full episode-id list positionally.

7 Confidence-Weighted Goal Distillation

Only a *hard-label* distillation term $\mathcal{L}_{\text{goal}}$ is implemented, not a soft-label KL term: the teacher returns exactly one discrete goal plus a scalar confidence, never a full probability distribution over the ontology, and fabricating a soft target from a single discrete choice was explicitly rejected by design.

$$\mathcal{L}_{\text{goal}} \;=\; \frac{1}{N}\sum_{i=1}^{N} w_i \cdot \big[-\log \hat{p}_\theta(g_i \mid s_i)\big], \qquad w_i = \text{clip}(c_i,\, w_{\min},\, 1), \quad w_{\min} = 0.05 \tag{1}$$

where $\hat{p}_\theta(\cdot \mid s_i) = \mathrm{softmax}(\ell_{\mathrm{goal}}(s_i))$ is the manager's subgoal head evaluated via a side-channel forward pass (`only_subgoal_logits=True`) that runs the manager GNN only – skipping the worker, value, and termination machinery entirely – and $c_i \in [0, 1]$ is the teacher's self-reported confidence for record i, clipped away from zero so a low-confidence label still contributes rather than being silently dropped.

Algorithm 3 Confidence-Weighted Goal Distillation Loss

function GoalDistillationLoss($\ell_{\mathrm{goal}} \in \mathbb{R}^{N\times 8}$, $g \in \{0, \ldots, 7\}^N$, $c \in [0, 1]^N$)
  $w \leftarrow \mathrm{clip}(c,\ 0.05,\ 1.0)$
  $\log \hat{p} \leftarrow \log\text{-softmax}(\ell_{\mathrm{goal}}, \mathrm{dim} = -1)$
  $\mathrm{nll}_i \leftarrow -\log \hat{p}_i[g_i]$ for each i
  return $\mathrm{mean}_i(w_i \cdot \mathrm{nll}_i)$
end function

An optional random-label control (`--llm_distill_shuffle_labels`, ablation condition C5) permutes `goal_idx` across records *in place* – preserving the real class-frequency distribution – so the same states and confidences train against structurally-scrambled labels, isolating whether an observed gain comes from real teacher knowledge or merely the regularizing effect of an extra auxiliary loss.

8 Optional Potential-Based Reward Shaping

A separate, independently togglable channel (`--llm_shaping`) applies potential-based reward shaping directly at the reward level, over the augmented (state, active-subgoal) space $x_t = (s_t, g_t)$:

$$F_t = \gamma\, \lambda_{t+1}\, \Phi(x_{t+1}) \;-\; \lambda_t\, \Phi(x_t) \tag{2}$$

using the *realized* next subgoal g_{t+1} (never a frozen copy of g_t), with terminal potentials fixed to 0 and shaping weight λ annealed across global training steps but held constant within a single PPO rollout window – so Eq. 2 reduces exactly to $\lambda \cdot (\gamma\Phi(x_{t+1}) - \Phi(x_t))$ within a window, an exact instance of the corrected potential-based shaping form (not an approximation of it), since λ's own update granularity is defined at the window level. Seven of the eight goals have a hand-authored, non-constant potential $\Phi_g(\text{state}) \in [0, 1]$ built only from the same partially-observable fields the baseline network already reads (fraction of hosts discovered, fraction scanned, fraction with user/root access, mean detection level, etc.); `RECOVER_OR_REPLAN` is left an explicit neutral placeholder ($\Phi \equiv 0$), documented rather than silently faked. All potentials are computed once at rollout-collection time under `torch.no_grad()` and cached into the reward *before* the PPO update, never recomputed during PPO's multi-epoch replay. **This channel was not enabled in the training run reported in Section 12** (`--llm_shaping` was not passed); it is documented here as part of the full system design and for architectural completeness.

Algorithm 4 Potential-Based Reward Shaping Term (per rollout window)

Require: rollout trace of length T (=`ppo_t`), each step's realized subgoal g_τ, done flag d_τ
Require: shaping weight λ_t (constant within this window)

for $\tau = 1, \ldots, T$ **do**
  $\Phi_\tau \leftarrow \Phi_{g_\tau}(s_\tau)$ ▷ gather the active goal's potential (Sec. 8)
end for
for $\tau = 1, \ldots, T$ **do**
  if $\tau < T$ **then**
    $\Phi'_{\tau+1} \leftarrow 0$ if d_τ else $\Phi_{\tau+1}$
    $F_\tau \leftarrow \lambda_t(\gamma\Phi'_{\tau+1} - \Phi_\tau)$
  else ▷ window-boundary step: $g_{\tau+1}$ not yet known
    $F_\tau \leftarrow \lambda_t(0 - \Phi_\tau)$ if d_τ else 0 ▷ exact if terminal, neutral otherwise
  end if
end for
return $\{F_\tau\}_{\tau=1}^{T}$ ▷ added to that step's env reward before `net.update`

9 Four-Stage Training Procedure

Training combines on-policy PPO over the environment reward with the distillation loss of Section 7 in four stages, controlled by a single schedule over global training step t (Eq. 3):

$$\lambda_{\text{goal}}(t) = \begin{cases} 0 & \text{if } t \geq (1-f)\,T_{\text{tot}} \quad \text{(Stage 4: RL-only, forced exactly 0)} \\ \lambda_0 \exp(-t/\rho) & \text{otherwise} \quad \text{(Stages 2–3: fixed-then-decaying joint weight)} \end{cases} \tag{3}$$

where $T_{\text{tot}} = \texttt{max\_epochs} \times \texttt{log\_rate}$ is the total planned step budget, f (flag `--llm_distill_rl_only_frac`) is the final fraction of training reserved as guaranteed RL-only, λ_0 (flag `--llm_distill_lambda_start`, default 1.0), and ρ (flag `--llm_distill_lambda_rate`, default 8000 steps). The $t \geq (1-f)T_{\text{tot}}$ branch exists because the plain exponential decay alone only *asymptotically* approaches zero and never guarantees a true RL-only phase within a bounded run – Stage 4 forces it exactly, once.

Algorithm 5 Full LLM-Distilled Training Procedure

Require: dataset $\mathcal{D}$ (Algorithm 2), warm-start epochs E_w, max epochs $E_{\max}$, schedule params (λ_0, ρ, f)

Stage 1 (warm-start): ▷ pure supervised, $\mathcal{L}_{\text{RL}} = 0$
for $e = 1, \ldots, E_w$ **do**
  for minibatch $B \subset \mathcal{D}$ (shuffled) **do**
    $\ell_{\text{goal}} \leftarrow \text{net}(B.\text{states}, \text{only\_subgoal\_logits=True})$
    $\mathcal{L} \leftarrow \textsc{GoalDistillationLoss}(\ell_{\text{goal}}, B.\text{goal\_idx}, B.\text{confidence})$
    backprop $\mathcal{L}$; clip grad norm; optimizer step
  end for
end for
hard-sync target network to warm-started weights (ρ_{target}=1.0)
Stages 2–4 (joint PPO + decaying/zeroed distillation):
for global step $t = 1, 2, \ldots$ **do**
  collect an on-policy rollout of length `ppo_t` using the current policy (Algorithm 1 each step)
  if `--llm_shaping` enabled **then**
    shape rollout rewards via Algorithm 4
  end if
  $\mathcal{L}_{\text{RL}} \leftarrow$ clipped-surrogate PPO loss over the rollout (Section 9.1)
  update net via $\mathcal{L}_{\text{RL}}$ ▷ primary gradient step
  $\lambda_{\text{goal}} \leftarrow$ Eq. 3 evaluated at t
  if $\lambda_{\text{goal}} > 10^{-6}$ **then**
    sample minibatch $B \subset \mathcal{D}$
    $\ell_{\text{goal}} \leftarrow \text{net}(B.\text{states}, \text{only\_subgoal\_logits=True}, \text{reset\_hidden=True})$
    $\mathcal{L}_{\text{goal}} \leftarrow \lambda_{\text{goal}} \cdot \textsc{GoalDistillationLoss}(\ell_{\text{goal}}, B.\text{goal\_idx}, B.\text{confidence})$
    backprop $\mathcal{L}_{\text{goal}}$ **as a separate gradient step** ▷ not fused into $\mathcal{L}_{\text{RL}}$'s backward
  end if
  every `log_rate` steps: evaluate, log explainability brief (Sec. 10), checkpoint
  if $t/$`log_rate` $\geq E_{\max}$ **then break**
  end if
end for

The distillation gradient step is deliberately kept *separate* from the PPO update rather than fused into one combined backward pass: fusing would require threading an auxiliary loss through the shared `rl.py:ppo()` update function used by every network architecture in this codebase, which this project's scope does not extend to. This alternating scheme optimizes the same two objectives, just not in a single backward pass.

9.1 PPO update

The manager's recurrence means PPO's usual multi-epoch importance-sampled replay (`ppo_k`= 3 epochs per rollout) must replay the *entire* `ppo_t`-step window in sequence from a saved hidden state h_{s_0}, not as an unordered minibatch, so the manager GNN's recurrent state stays consistent with the actions being replayed. Advantage is computed against a soft target network updated by Polyak averaging ($\rho_{\text{target}} = 0.1$), and the clipped surrogate objective $\mathcal{L}_\pi = -\mathbb{E}\big[\min(\rho_t A_t, \text{clip}(\rho_t, 1{-}\epsilon, 1{+}\epsilon)A_t)\big]$ (ϵ=0.2) is combined with a value-regression term (only over steps where the agent chose to continue rather than terminate, when `-use_a_t` is set) and an

entropy bonus, both on separate coefficients α_v, α_h decayed on their own schedules independent of λ_{goal}.

10 Live LLM-Explainability Monitoring

A per-epoch, always-on (not gated behind a verbosity flag) console report – *the LLM Explainability Brief* – surfaces two categories of live diagnostic signal that would otherwise only be visible by manually inspecting tensors:

1. **Manager subgoal-selection histogram.** Every realized subgoal selection during the epoch's rollouts (`raw_a[2]` from Algorithm 1) is accumulated into an 8-bin histogram and printed as a percentage breakdown against the fixed ontology names – but *only* when `--llm_distill` is active, since only then is the subgoal head being trained toward that specific name binding; without distillation, the same histogram is explicitly labeled as anonymous positional-index counts rather than silently mislabeled with names the policy was never trained to respect.

2. **Distillation/shaping diagnostics.** When `--llm_distill` is active: the current $\lambda_{\text{goal}}(t)$ (Eq. 3), the epoch's mean $\mathcal{L}_{\text{goal}}$ (or an explicit "$\lambda_{\text{goal}} \approx 0$ this window" note rather than a stale number), and the dataset size in use. When `--llm_shaping` is active: the shaping weight λ_t and the ratio $|F_t|/|r_{\text{env},t}|$ – a direct, checkable signal for whether the shaping term is contributing a measurable gradient relative to the raw environment reward, rather than inferring a "no measurable effect" outcome only from final performance.

Example output (this project's own training run, epoch 22 of 200):

```
[LLM EXPLAINABILITY BRIEF] Epoch 22
  Manager subgoal selections this epoch:
    DISCOVER_SUBNET 91.8% (93971)
    ENUMERATE_HOST 6.9% (7039)
    GAIN_INITIAL_ACCESS 1.0% (1056)
    ESCALATE_PRIVILEGE 0.0% (15)
    PIVOT 0.1% (97)
    CAPTURE_SENSITIVE_HOST 0.2% (154)
    REDUCE_DETECTION 0.1% (67)
    RECOVER_OR_REPLAN 0.0% (1)
  llm_distill (manager-level): lambda_goal=0.9114 mean L_goal=0.4629 dataset=2000 records
```

Listing 1: Explainability brief, early training (11% through the run)

```
[LLM EXPLAINABILITY BRIEF] Epoch 200
  Manager subgoal selections this epoch:
    DISCOVER_SUBNET 100.0% (102400)
    ENUMERATE_HOST 0.0% (0)
    GAIN_INITIAL_ACCESS 0.0% (0)
    ESCALATE_PRIVILEGE 0.0% (0)
    PIVOT 0.0% (0)
    CAPTURE_SENSITIVE_HOST 0.0% (0)
    REDUCE_DETECTION 0.0% (0)
    RECOVER_OR_REPLAN 0.0% (0)
  llm_distill (manager-level): lambda_goal=0.0000 mean L_goal=n/a (lambda_goal ~ 0 this window)
      dataset=2000 records
```

Listing 2: Same run, epoch 200 (final) – subgoal-selection collapse

This is not a cosmetic feature: it is the mechanism that made the subgoal-selection collapse analyzed in Section 12.2 directly observable during training, in human-readable form, epoch by epoch – rather than requiring a post-hoc statistical trawl through the raw JSONL step log to notice it. A curriculum status banner (network realism-stage parameters: IDS enabled/decay/thresholds, scan noise false-positive/negative rates, network-reliability timeout probability, service churn probability) is printed alongside it at early epochs, every 10th epoch, and at scenario-declared curriculum stage transitions, giving a complete picture of both *what the environment is doing to the agent* and *what the agent's manager is choosing to do* at every reported point in training.

11 Experimental Setup

Table 3: Training run configuration (run id `b33undf9`).

Scenario (training)	`corp_100hosts_dynamic`{base,_varA,_varB}.v2.yaml (mixed)
Test scenario (in-loop `eval_tst`)	`corp_100hosts_dynamic`.v2.yaml (base)
Architecture	`NASimNetDHRL` (212,640 params), `graph_v2` obs., `-use_a_t`
Episode step limit	400
Parallel environments (`-batch`)	128, over 8 CPU workers
PPO	γ=0.99, ppo_k= 3, ppo_t= 8, ϵ=0.2, α_v=0.0333
Learning rate	7×10^{-4}, decayed $\times$0.8 every 10,000 steps, floor 3×10^{-4}
Entropy coeff. α_h	0.02, time-decayed every 15,000 steps ($\times$0.5), floor 0.005
Epoch length / max epochs	100 steps/epoch, 200 epochs (20,000 train steps total)
LLM distillation	enabled; warm-start E_w=3 epochs; ρ=8000; f=0.1 (Stage 4 from step 18,000)
Teacher dataset	`training_data/llm_teacher_dataset` (unsplit, all valid records)
Seed	1
Device	CPU (8 threads/process, BLAS pinned to 1 thread/process)

11.1 Teacher dataset

The dataset used for distillation was collected via Algorithm 2 against the single-model LLM backend (`qwen2.5:3b-instruct`). Table 4 summarizes the audited dataset actually consumed by the training run in Table 3.

Table 4: Teacher dataset audit (`llm_teacher/audit_dataset.py`, this project's dataset).

Total records	2,235
Valid	2,000 (89.5%)
Rejected	235 (10.5%)
– executable text in rationale	124
– horizon out of range	105
– API error (HTTP 500)	6
Distinct episodes	170
Train / test split (episode-level)	2,000 / 235 records
Teacher backend/model	`qwen2.5:3b-instruct` (100% of records)
Total labeling time	17.1 min (0.46 s/record mean)

Table 5: Valid-record class balance and trigger mix – both show the same imbalance the collapse in Section 12.2 tracks.

Goal class	Count	Mean conf.	Trigger	Count
DISCOVER_SUBNET	1683	0.83	first_compromise	805
ENUMERATE_HOST	233	0.70	periodic	559
GAIN_INITIAL_ACCESS	55	0.94	high_student_entropy	439
ESCALATE_PRIVILEGE	4	0.96	new_subnet_discovered	215
CAPTURE_SENSITIVE_HOST	12	0.92	episode_start	170
REDUCE_DETECTION	9	0.95	privilege_change	39
PIVOT	4	0.84	three_consecutive_failures	8
RECOVER_OR_REPLAN	0	n/a		

Five of eight classes fall below the audit tool's default minimum-class-count threshold (20); RECOVER_OR_REPLAN has *zero* valid labeled examples in this dataset. This is flagged by audit_dataset.py itself at collection time ("training the distillation head against these classes now will mostly be noise") and is directly relevant to Section 12.2.

12 Results

12.1 Training dynamics

Table 6: Selected training-log snapshots (training_data/runs/b33undf9.json). eval_tst is computed on the single base scenario every logged epoch.

Train step	Loss	Entropy	eval_tst reward_avg_ep.	eval_tst eplen_avg	eval_tst captured_avg
100	1.98	5.43	–	400.0	24.8
1000	8.84	3.95	–	400.0	28.9
14400	2.49	7.37	270.7	400.0	31.1
19800	–	–	–	115.2	29.6
20000 (final)	2.91	4.67	289.4	105.9	30.0

12.2 Subgoal-selection collapse

The manager subgoal-selection histogram (Section 10) already showed 91.8% DISCOVER_SUBNET at epoch 22 (11% through training); by the final epoch this reached **100.0%**, with all seven other named goals – including CAPTURE_SENSITIVE_HOST, the goal most directly tied to the task's objective – at exactly zero selections. This mirrors the teacher dataset's own class imbalance almost exactly (84.2% DISCOVER_SUBNET in Table 5): a manager repeatedly distilled toward a heavily DISCOVER_SUBNET-dominated target, combined with the persistence *switch* gate (Section 3) having no explicit pressure against staying put, is a plausible, evidence-consistent mechanism for the collapse, though this report does not run the random-label control (condition C5, Section 7) needed to establish it as the sole cause rather than a contributing one. Despite the collapse in the *labeled* subgoal signal, the worker continued to make tangible task progress (captured_avg $\approx$ 30 hosts throughout), consistent with the worker's action policy retaining useful behavior largely independently of which subgoal label the manager was nominally emitting at a given moment.

12.3 Checkpoint-selection pitfall

By default (`-save_best_metric captured_avg`), the "best" checkpoint (`model_best.pt`) is saved whenever `eval_tst.captured_avg` sets a new running maximum on the single, fixed, easiest training scenario. In this run, that metric peaked at train step 14400 (Table 6) and was never strictly exceeded again – even though `eval_tst.eplen_avg` dropped from 400.0 (every episode exhausting the step budget) to 105.9 by the final step, meaning the policy had learned to finish episodes roughly 3.8× faster for a comparable capture count. `captured_avg` alone cannot see an episode-length improvement, so `model_best.pt` silently froze at a materially worse-behaving policy than the run's own final checkpoint, `model.pt`.

12.4 Cross-scenario generalization

Both checkpoints were evaluated on all five scenario instances (Section 2), 100 held-out episodes per scenario, greedy (`net.eval()`) action selection.

Table 7: `model_best.pt` (frozen at train step 14400) vs. `model.pt` (final, train step 20000). `reward_avg_episodes` is the sum of per-episode reward, not the (misleading, unequal-episode-length) per-step average.

	`model_best.pt`			`model.pt` (final)		
Scenario	reward_avg_ep.	eplen	captured	reward_avg_ep.	eplen	captured
base (train)	3.24	400.0	30.2	**259.1**	**111.6**	29.6
varA (train)	21.8	400.0	17.5	**80.5**	**271.2**	17.5
varB (train)	−12.2	400.0	14.4	**46.0**	**272.7**	13.3
test (held-out)	−36.7	399.4	11.9	**−8.3**	**277.8**	9.2
bridge (held-out)	−22.1	400.0	25.9	**205.5**	**117.9**	24.7

`captured_avg` is essentially flat between the two checkpoints on every scenario (within evaluation noise over 100 episodes) – the final checkpoint is *not* trading capture ability for speed, it is achieving the same capture ability in far fewer wasted steps. `reward_avg_episodes` flips from negative to strongly positive on 3 of 5 scenarios and improves substantially on the remaining two (including the held-out `test` scenario, still the weakest under both checkpoints, consistent with it having the lowest mean exploit probability in Table 1). The practical conclusion is direct: for this architecture and this choice of `-save_best_metric`, the plain final checkpoint generalizes measurably better across held-out scenarios than the metric-selected "best" one, and `reward_avg_episodes` (which penalizes an episode that never terminates) would be the more appropriate optimization target for `-save_best_metric` in future runs of this system.

12.5 Comparison against a non-distilled Pen-DHRL baseline

Beyond the checkpoint-selection question above, Pen-DHRL-TD's final checkpoint was compared against a plain `NASimNetDHRL` baseline trained with *no* LLM distillation – identical architecture and parameter count (212k), isolating the distillation intervention itself. The baseline was trained twice independently; the value reported per scenario is from its stronger run, so it is shown in its best light rather than penalized by an unlucky run. All four evaluation metrics are reported for both, not just `reward_avg_episodes`. No baseline run was collected for the held-out `test` scenario, so it is omitted here (Pen-DHRL-TD alone on `test` is already in Table 7 above).

Table 8: Pen-DHRL-TD (final checkpoint).

Scenario	reward_avg	reward_avg_episodes	eplen_avg	captured_avg
base	2.3216	259.12	111.61	29.63
varA	0.2967	80.47	271.18	17.50
varB	0.1688	46.04	272.69	13.25
bridge	1.7425	205.48	117.93	24.67

Table 9: Pen-DHRL baseline – same architecture, same parameter count, no `--llm_distill`.

Scenario	reward_avg	reward_avg_episodes	eplen_avg	captured_avg
base	0.5428	217.11	400.0	29.59
varA	0.1708	68.33	400.0	13.73
varB	0.0778	31.12	400.0	9.39
bridge	1.1470	161.96	141.20	23.36

Pen-DHRL-TD vs. Pen-DHRL baseline (100 held-out episodes/scenario)

Pen-DHRL-TD (final checkpoint, LLM-distilled)

Scenario	reward_avg	reward_avg_episodes	eplen_avg	captured_avg
corp_100hosts_dynamic (base)	2.3216	259.12	111.61	29.63
corp_100hosts_dynamic_varA	0.2967	80.47	271.18	17.50
corp_100hosts_dynamic_varB	0.1688	46.04	272.69	13.25
corp_100hosts_dynamic_bridge	1.7425	205.48	117.93	24.67

Pen-DHRL baseline (same architecture, same param count, no distillation)

Scenario	reward_avg	reward_avg_episodes	eplen_avg	captured_avg
corp_100hosts_dynamic (base)	0.5428	217.11	400.0	29.59
corp_100hosts_dynamic_varA	0.1708	68.33	400.0	13.73
corp_100hosts_dynamic_varB	0.0778	31.12	400.0	9.39
corp_100hosts_dynamic_bridge	1.1470	161.96	141.20	23.36

Figure 1: Both tables above, rendered directly from the evaluation output.

Pen-DHRL-TD wins on every metric (`reward_avg`, `reward_avg_episodes`, `captured_avg`) on every scenario, and terminates well before the 400-step limit on all four, while the baseline only does so on `bridge`. Because the two runs share architecture and parameter count exactly, this isolates the LLM-teacher distillation signal itself as the source of the gain, rather than any difference in model capacity.

13 Discussion and Limitations

- **Class imbalance in the teacher dataset.** Five of eight ontology classes are below a reasonable minimum count for reliable supervision, and `RECOVER_OR_REPLAN` has zero examples (Table 5). The trigger set (Algorithm 2) over-samples early-episode/ exploration-phase states relative to rare escalation/pivot moments; a targeted collection pass using `--policy checkpoint`

against a partially-trained agent that has already learned to reach those rarer states (rather than the `random`/`untrained_net` policies that dominate easy states) is the documented mitigation path.

- **Distillation-collapse hypothesis, not proof.** Section 12.2's causal link between dataset imbalance and the subgoal-selection collapse is plausible and evidence-consistent but not isolated from the persistence-gate dynamics or the RL objective's own contribution. The random-label control (`--llm_distill_shuffle_labels`, condition C5) and the heuristic-teacher ablation (Section 5) both already exist in this codebase to disambiguate this and were not run for the present report.
- **Single teacher model.** All reported results use one 3B-parameter local model. The cascade escalation path (Section 5) and workstation-scale larger-model candidates are implemented but unevaluated here.
- **Potential-based shaping unused.** Section 8's reward level channel was not enabled for this run; its interaction with the collapse observed in distillation-only training is unexplored.
- **Checkpoint-selection metric.** As shown directly in Table 7, `-save_best_metric captured_avg` is not a safe default for this task; it is blind to episode-length degeneracy.
- **Baseline comparison is one run.** Table 9 reports the non-distilled baseline's stronger of two independently trained runs, not a distribution over many runs – a useful signal, not a statistically rigorous claim of significance. The same caveat applies symmetrically to Pen-DHRL-TD itself: the results throughout this report come from a single training run.

14 Conclusion

Pen-DHRL-TD demonstrates a complete, offline-only pipeline for grounding a hierarchical RL agent's otherwise-anonymous subgoal space in LLM-derived semantic supervision: a fixed goal ontology, a sanitized/validated/provenance-tracked teacher dataset built via triggered sampling under a diversified policy mixture, a confidence-weighted distillation loss folded into a four-stage training schedule alongside standard PPO, and a live explainability mechanism that surfaces the manager's realized behavior every epoch rather than only at evaluation time. Applied to a 100-host dynamic corporate scenario family, this pipeline surfaced concrete, actionable findings that would not have been visible from aggregate reward curves alone: a subgoal-selection collapse traceable to the teacher dataset's own class imbalance, and a checkpoint-selection metric blind to episode-length degeneracy that caused the saved "best" model to substantially underperform the run's own final policy on held-out generalization. Set against an independently trained, same-architecture, same-parameter-count Pen-DHRL baseline run without the teacher signal, the final distilled policy also wins on every reported metric on every scenario tested – evidence that the gain traces to the distillation signal itself rather than model capacity. All of these findings, and the tooling that surfaced them, are directly actionable for the next data-collection and training pass.